

MSXPi Quick Start Guide

Version 1.6

Real hardware (MSXPi interface + Raspberry Pi) and openMSX. Everything else is in the Users and Developers Guide.

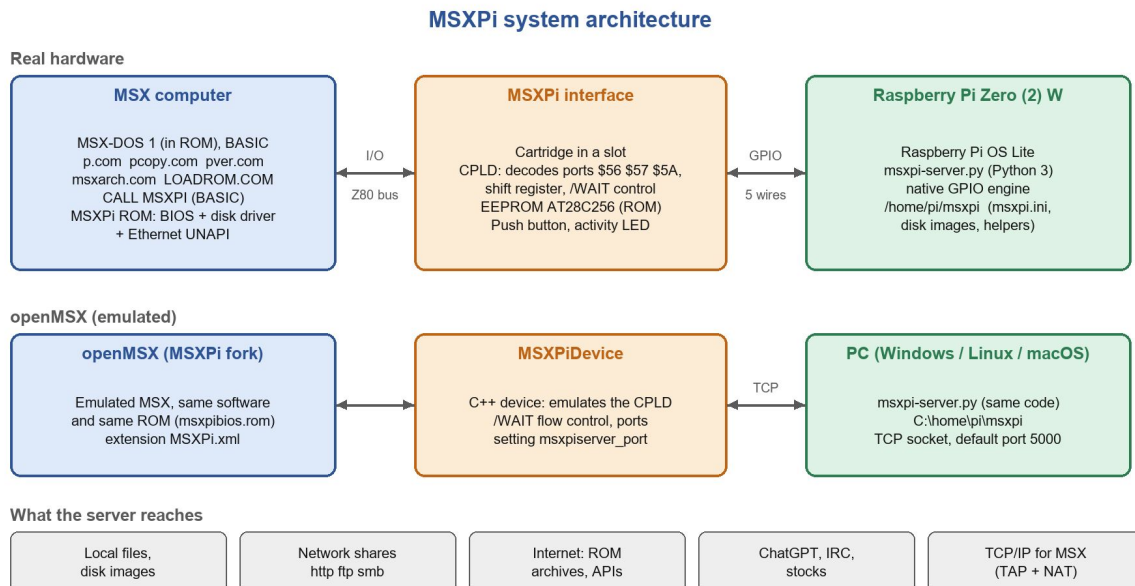


Figure 1: What you are setting up

Read this first: Version 1.6 breaks compatibility with earlier ROMs and servers. Upgrade the ROM, the msxpi-server, the client commands (.COM files) and, on real hardware, the CPLD firmware together. A v1.6 ROM on older CPLD firmware still works, only slower.

Pick your path: **Part 1** for a real MSX with an MSXPi interface, **Part 2** for openMSX with no hardware. Each step ends with a check - do not go on until the check passes.

Commands typed on the MSX are shown as `C:\>` lines or in a grey box; the same commands work on hardware and in openMSX.

1 Part 1 - Real hardware

1.1 What you need

- The MSXPi interface with an AT28C256 EEPROM. Build notes: [documents/MSXPi Assembly Instructions.txt](#).
- A Raspberry Pi Zero W or Zero 2 W (a Pico does not work) with its 40-pin header on the side that matches your PCB, and a micro SD card of 8 GB or more.
- An MSX with a disk or SD interface (MSX-DOS 1, MSX-DOS 2 or Nextor) and a way to copy files to it from a PC. MSXPi's own ROM contains MSX-DOS 1 only: Nextor or MSX-DOS 2 must come from that other disk interface, which MSXPi coexists with - the best setup.
- Only if the CPLD is not programmed yet: an Altera USB-Blaster and Quartus 13.0 SP1 on Windows.
- The files in this repository: [software/target](#) (programs and ROM), [hardware/CPLD_Project](#) (firmware).

1.2 Step 1 - Program the CPLD (only if new or older than v1.6)

1. Plug the interface into the MSX, connect the USB-Blaster to the JTAG connector, switch the MSX on.
2. In Quartus: Tools, Programmer, mode JTAG, Add File, choose the [.pof](#) for your board, tick Program/Configure and Verify, Start.

Board	File in hardware/CPLD_Project
V1.3 Rev.1 (built for v1.3 to v1.5)	MSXPi_v1.6.pof
V1.1 Rev.1, V1.2.1b	v1.1/MSXPi_v1.6_wait.pof
v0.8.2 / v1.0 (PCB v0.7 Rev.7)	v0.8.2/MSXPi_v1.6_polled.pof

Check: in BASIC, [PRINT INP\(&H57\)](#) must not print 254 or 255 (no CPLD answered); [pver](#) shows the same build ID in readable form later. Older boards need extra settings later (see the Legacy Support document).

1.3 Step 2 - Write the ROM to the EEPROM

1. Copy [at28c256.com](#) and [msxpibios.rom](#) from [software/target](#) to an MSX disk. The interface must be plugged in; the Pi is not needed yet.
2. If the interface already boots a ROM, remove its enable jumper (SLTSL) first.
3. Boot MSX-DOS and list the slots: [at28c256 /i](#). Write down the slot of the interface.
4. Write the ROM (slot 2 in this example): [at28c256 /s 2 msxpibios.rom](#). Add [/r](#) to write slowly if your chip fails. The chip is write-protected afterwards.

5. Power off and set the jumpers. **With another disk interface (recommended): remove SLTSL.**
To boot MSX-DOS 1 from a disk image on the Pi: close SLTSL and select the bank with the MSXPi ROM.

Check: with the enable jumper closed, `CALL MSXPIVER` in BASIC prints the version.

1.4 Step 3 - Prepare the Raspberry Pi

Write "Raspberry Pi OS Lite (32-bit)" with Raspberry Pi Imager (user pi, SSH on, your WiFi entered), boot the Pi, log in and run:

```
wget https://tinyurl.com/MSXPi-Setup
chmod 755 MSXPi-Setup
bash ./MSXPi-Setup
```

The script checks the Pi, the network and the clock, installs Python 3 with the modules `requests`, `fs` and `RPi.GPIO` (`fs` needs `pkg_resources`, so it also pins `setuptools` below version 81), the packages for archives, audio and networking, builds the native GPIO engine, creates the MSXPi home directory `/home/pi/msxpi` (server, `msxpi.ini`, disk images, helpers), starts the server, checks that it runs, and reboots - a few minutes, not the hour earlier versions could take.

It asks which board you have; skip the question with `--board v1.3`, `--board v1.1` or `--board old`, and see `--help` for the rest (WiFi, `--no-reboot`, ...). It is safe to run again later: it updates the server and repairs what is missing, and never overwrites your `msxpi.ini`.

In `msxpi.ini` a board **without** the shutdown push-button needs `var RPI_SHUTDOWN=none`. Optional keys: OPENAIKEY (ChatGPT), and the stock quote keys.

Insert the card, plug the Pi into the interface **component side facing you**, switch the MSX on. The interface LED lights when the server runs.

1.5 Step 4 - Copy the programs to the MSX

Copy everything in `software/target` (at least `p.com`, `pcopy.com`, `pver.com`, `msxarch.com`, `msxarch.ini`, `LOADROM.COM`, `INL.CFG`) to your MSX disk or SD card.

```
pver
```

Check: `pver` prints the board, the CPLD build, `/WAIT: yes (burst)` and the server version.

"Connection error"? The Pi may still be booting (first boot: up to three minutes), or the versions do not match. Press ESC, wait five seconds, try again.

1.6 Step 5 - WiFi and network

```
p set WIFISSID Your Wifi Name
p set WIFIPWD YourWifiPassword
p wifi set
p reboot
```

Then, for TCP/IP programs (the Ethernet UNAPI is in the ROM; do not run RAMHELPR):

```
MSR I          (MSX-DOS 1 only)
INL I
HOST GOOGLE.COM
```

Check: `HOST` prints an address. If not: `p netreset`. "DNS not found" means the resolver: use `INL IP P <address>`.

1.7 Step 6 - Have fun

```
p dir /home/pi/msxpi
| pcopy m:pver.com
| msxarch
p chatgpt Tell me a fun fact about MSX
```

Update later with `msxpiupd.bat` (MSX files), the setup script (server) and `at28c256` (ROM). Run ``p shut`` before switching the MSX off.

2 Part 2 - openMSX

2.1 What you need

- **openMSX with the MSXPi device.** The official openMSX is <https://github.com/openMSX/openMSX>; the MSXPi device is in the fork - download the newest release for your system from <https://github.com/costarc/openMSX/releases> (read its release notes).
- Python 3.9 or newer, with `python -m pip install requests fs "setuptools<81"`.
- **7-Zip** on the PATH (zip, lzh, pma, 7z archives).
- The MSXPi home directory: `/home/pi/msxpi` (on Windows `C:\home\pi\msxpi`).
- The files from this repository: the server in `software/Server/Python/src`, the programs in `software/target`.

Windows shortcut: run `software/Server/Setup/msxpi-windows-setup.ps1` (right-click, Run with PowerShell). It installs Python, 7-Zip, the modules, the home directory, openMSX with MSXPi, the TAP driver and a launcher with a desktop shortcut. Then skip to step 4.

2.2 Step 1 - Create the home directory

Put in it: `msxpi-server.py`, `mapper_detect.py` and every `msxpi_*.py` module from `software/Server/Python/src`, `msxpi.ini` (copy `msxpi-JumperLeft.ini` and rename it) and a disks folder with `msxpiboot.dsk` and `tools.dsk`. On Windows create `C:\home\pi\msxpi` and start the server from drive C: - the server path resolves on the current drive (or set `MSXPI_HOME` to another folder).

2.3 Step 2 - Start the server

```
cd C:\home\pi\msxpi
python msxpi-server.py
```

It listens on TCP port 5000 and prints every command it receives. Leave it running.

2.4 Step 3 - Prepare a disk folder

Make a folder `FloppyA`. Copy `MSXDOS.SYS` and `COMMAND.COM` (from `software/target/disks`) and all the MSXPi programs from `software/target` into it.

2.5 Step 4 - Start openMSX

```
| openmsx -machine Panasonic_FS-A1WSX -ext MSXPi -diska FloppyA
```

Check: at the DOS prompt type `pver`. It should show `/WAIT: yes (burst)` and the server version. "Connection error": the server is not running, the ports differ, or the versions do not match.

Optional - another TCP port: in the openMSX console (F10) type `set msxpiserver_port 5001` and start the server with the environment variable `MSXPI_PORT=5001` to match. Default: 5000.

2.6 Step 5 - Network in the emulator (optional)

Windows: install the OpenVPN TAP driver, run `powershell -ExecutionPolicy Bypass -File msxpi-tcpip-setup.ps1` as **Administrator** (again after every reboot), start the server, then on the MSX `MSR I` (MSX-DOS 1 only), `INL I`, `HOST GOOGLE.COM`. If the server prints "falling back to MockLink", something else has the adapter open.

3 Cheat sheet

Command	Does
p ver	Board, CPLD, /WAIT and server version
p dir [path]	List Pi, network or r1: r2: m: locations
p cd [path]	Set / show the current path
pcopy src [dest]	Copy Pi/network to MSX (/z unpacks); pcopy a:f /path copies to the Pi
p run cmd	Run a command on the Pi (: : for a pipe)
p set NAME value	Change msxpi.ini variables (WIFISSID, OPENAIKEY, ...)
p wifi [set], p wlanreset, p netreset	WiFi, WiFi reset, rebuild the network for the MSX
p date	Set the MSX clock from the Pi
p reboot / shut / restart	Reboot the Pi / shut it down / restart the server
msxarch	Browse ROM repositories and start a game
LOADROM name /N	Load a ROM from the network
p chatgpt text	Ask ChatGPT (needs OPENAIKEY)
ESC	Abort; wait 5 seconds before the next command

4 When something goes wrong

Symptom	Try
"Connection error" from pver or p	Pi still booting (wait 3 min on the first boot); ESC, wait 5 s, retry; run the server by hand and read its messages; check that ROM, server and .COM files are all v1.6
pver: Board: no MSXPi found	CPLD not programmed, or the interface is not seated; check <code>PRINT INP(&H57)</code>
pver: /WAIT: no (polled)	Older CPLD firmware or a board without /WAIT: works, but slower
Pi reboots by itself (older boards)	Add <code>var RPI_SHUTDOWN=none</code> to msxpi.ini
INL: No mapper support routines found	RAMHELPR was run before MSR: cold boot, then MSR and INL only
HOST: DNS not found	<code>INL IP P <resolver address>;</code> run <code>p netreset</code>
MSX-DOS 1 does not boot from the Pi	Wait for the Pi to boot; ESC to skip to BASIC; replace <code>msxpiboot.dsk</code>
openMSX: nothing happens	Start msxpi-server.py first; port must match <code>msxpiserver_port</code> ; use the fork build of openMSX

More: README.md in the repository, the MSXPi Users and Developers Guide, documents/Legacy Support.odt for older boards, and the Hard Way Raspberry Pi tutorial (English and Portuguese). Issues: <https://github.com/costarc/MSXPi/issues>